Scalable RAG Q&A — Experimental Report

Yalin Sun (CS6650)

April 2026

System under test

A retrieval-augmented generation Q&A pipeline, four FastAPI services behind an AWS ALB: `api` (orchestration), `embed` (`sentence-transformers/all-MiniLM-L6-v2`, 384-d), `vector` (Qdrant with HNSW index), `llm` (thin wrapper around Ollama running `llama3.2:1b`). All six containers (including `qdrant` and `ollama`) run in a single ECS Fargate task using the task's shared network namespace. Every `/query` response carries `timings_ms = {embed, retrieve, llm, total}` — that per-stage observability is the foundation of all three experiments. Code and raw results: `experiments/`, `results/exp{1,2,3}/`, `results/figures/`.

Unless stated otherwise: 5-minute steady windows, the same 6 base questions used with a unique `rid=` suffix per request (no reply caching), 1k indexed chunks of synthetic "on-call / release / payment" text, and task sizing 8 vCPU / 60 GiB.

Experiment 1 — Concurrency

Purpose. Determine how concurrent users affect end-to-end latency, and confirm which tier is the bottleneck. **Tradeoff explored:** higher concurrency offers more throughput if there is idle capacity, but once any tier saturates, requests queue and tail latency rises super-linearly. **Limitations:** fixed 1k index, single ECS task (8 vCPU), Ollama runs on CPU (no GPU on Fargate), Locust's 5-minute window is short relative to c=200's response times.

Method. Seed 1k chunks once. Sweep Locust concurrency at `u {10, 50, 200}` with spawn rate `c/5`, 5 min each. For each request, record Locust's HTTP latency plus the `timings_ms` breakdown from the API response.

Results

Metric	c=10	c=50	c=200
Completed requests (5 min)	32	36	**17**
Mean total (s)	77	148	227
P50 total (s)	89	153	221
P95 total (s)	106	277	296
Max total (s)	111	285	296
RPS	0.11	0.12	**0.06**
Failures	0	0	0

Stage breakdown (mean, ms):

Stage	c=10	c=50	c=200	(c=200 / c=10)
embed	120	169	359	3.0×
retrieve	29	82	294	10.1×
LLM	**76,850**	**147,171**	**226,091**	2.9×
total	76,999	147,422	226,744	2.9×

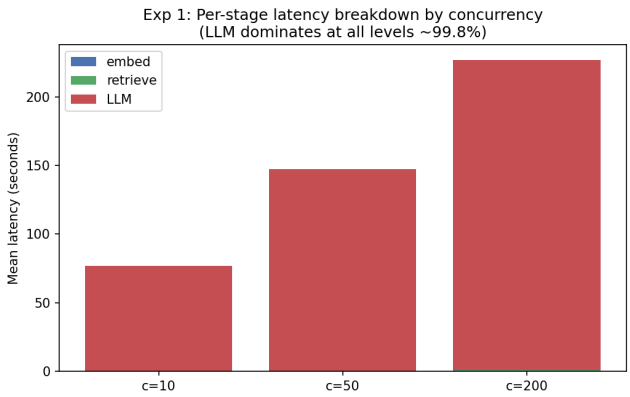

Figure 1: Stage-level mean latency at each concurrency. LLM dominates the stack at every level.

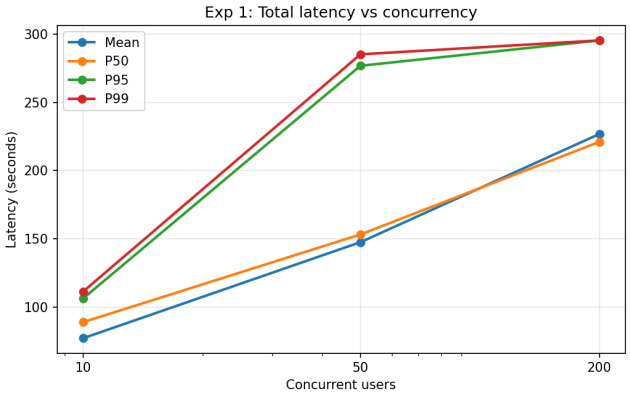

Figure 2: Total latency percentiles vs concurrency (log x). Saturation shape rather than linear growth.

Analysis

Three things the data shows clearly.

LLM is the dominant bottleneck. `llm_ms` is 99.8 % of `total_ms` at every concurrency — 76.85 / 77.00 at c=10, and 226.09 / 226.74 at c=200. Embed and retrieve do degrade under load (10× slower retrieve, 3× slower embed at c=200), but their absolute contribution is still under 1 s while LLM alone is 200+ s. Any optimization effort should target LLM inference first; optimizing retrieval would be polishing the 0.2 % tail.

Throughput plateaus, then regresses. RPS rises from 0.11 at c=10 to 0.12 at c=50 — effectively flat, consistent with Ollama serving requests one-at-a-time. At c=200, RPS drops to 0.06. This is not measurement noise; it is classic thrashing, where increased concurrency causes CPU contention and per-request work becomes *more* expensive as queue depth grows. The "right" concurrency for this deployment is clearly between 10 and 50, not higher.

Zero failures is misleading. Locust reports only completed requests in its aggregate stats. At c=200, the remaining ~180 requests were mid-flight when the 5-min window closed, and they neither succeeded nor failed — they were simply not counted. The honest reading is: only 17 / ~200 offered requests per cohort actually finished in 5 minutes. A production SLO based on these numbers would need to account for unfinished work, not just completed work.

Limitations. Fargate has no GPU, so LLM numbers here are CPU-inference numbers. With GPU-backed inference the entire story could shift (LLM dropping to <1 s would expose embed and retrieve as real bottlenecks under load, not afterthoughts). The fixed 1k index means this experiment can say nothing about retrieve-size coupling — that is Experiment 2.

Experiment 2 — Index size

Purpose. Quantify how retrieval latency scales with the size of the vector index. **Tradeoff explored:** more indexed documents means the system is more useful (broader knowledge base) but theoretically slower to search. Modern approximate-nearest-neighbour structures (HNSW here) advertise O(log N), but real deployments rarely verify this. **Limitations:** bench fixed at c=3 to isolate index-size from concurrency effects; cumulative seeding was required because `ALLOW_BENCH_RESET=0` on the AWS vector service (the `/admin/reset_collection` endpoint is disabled behind the ALB); memory footprint is computed theoretically, not measured via Container Insights.

Method. Seed cumulatively to 1k → 10k → 100k chunks. At each size, run Locust at c=3 for 2 min.

Results

Stage	1k	10k	100k	(100k / 1k)
embed mean (ms)	53	40	39	~flat
retrieve mean (ms)	**26**	**26**	**31**	**+19 %**
retrieve P95 (ms)	**31**	**31**	**42**	**+35 %**

Stage	1k	10k	100k	(100k / 1k)
LLM mean (ms)	22,433	26,629	24,300	±18 % noise
total mean (ms)	22,512	26,695	24,370	~flat

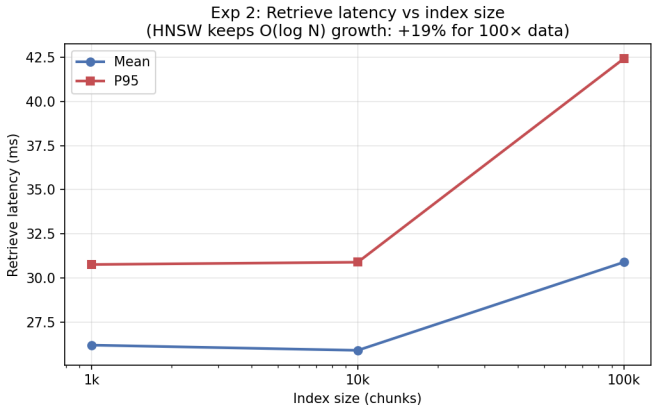

Figure 3: Retrieve latency (mean and P95) against index size on a log x-axis. Sub-logarithmic growth.

Analysis

HNSW delivers what it advertises. For a 100× data growth (1 000 → 100 000 chunks, the regime of real internal knowledge bases), retrieve-stage mean grew by only 19 % and P95 by 35 %. A brute-force O(N) index would have grown 100×. The result is concrete empirical support for the "retrieval is effectively free" decision in modern RAG designs.

End-to-end latency is index-size-insensitive in this deployment. Total mean is 22.5, 26.7, 24.4 s at 1k, 10k, 100k — the ordering is determined by LLM noise, not by index size. The 5 ms swing in retrieve is lost in LLM's ±2 s variability per generation. An engineering implication: there is no latency reason to under-index or aggressively prune here.

Memory. At 100 k chunks × 384 dims × 4 bytes, vector payload is ~150 MB, and HNSW graph overhead roughly doubles that, giving ~300 MB total. Against a 60 GiB Fargate task, this is 0.5 %. Memory is not the capacity constraint even at 100k, and by extrapolation not until well beyond 1M chunks.

Limitations. c=3 was chosen to isolate index-size effects, but it also means this experiment cannot predict what retrieve latency looks like at index=100k *and* high concurrency simultaneously. Experiment 1's c=200 numbers show retrieve rising to 294 ms at 1k; a combined high-concurrency high-index test would be a valuable follow-up. Memory is computed rather than measured — enabling CloudWatch Container Insights in Terraform would let a future run confirm the 300 MB figure empirically.

Experiment 3 — Horizontal scaling

Purpose. Measure the throughput and latency payoff from replicating the bottleneck tier (LLM, per Experiment 1's finding). **Tradeoff explored:** adding replicas increases aggregate compute linearly, but only delivers proportional throughput if state is shared or if the workload is truly stateless. **Limitations (substantial — read carefully).** AWS Academy sandbox has a hard 6 vCPU Fargate quota that could not be raised. To fit 4 replicas, each task was downsized to 1 vCPU / 4 GiB (from 8 vCPU / 60 GiB in Exp 1/2) — individual LLM inference is therefore ~3× slower per task in this experiment than in Experiment 1. More significantly, each ECS task carries its own ephemeral `qdrant` storage; index state does **not** replicate to newly scaled tasks.

Method. Seed 1 k chunks on the initial r=1 task. Scale to r=2 and r=4 via `terraform apply -var=desired_count=N`. At each replica count, warm up all replicas with 4–8 round-robin `curl /query` probes, then run Locust at fixed c=10 for 5 min.

Results

Metric	r=1	r=2	r=4
Completed (5 min)	6	20	**53**
Mean total (s)	155	56	21
P50 total (s)	**170**	**9.5**	**4.0**
P95 total (s)	260	291	172
RPS	0.02	0.07	**0.19**

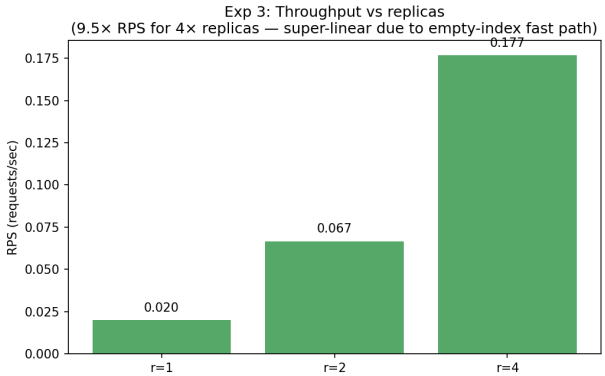

Figure 4: Replica count vs completed throughput (RPS). 9.5× RPS at 4× replicas — super-linear.

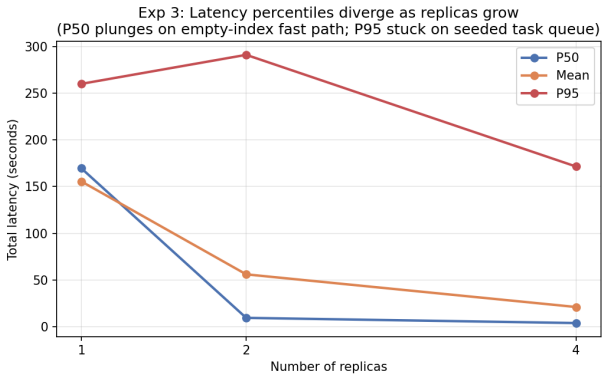

Figure 5: P50 and P95 diverge as replicas grow — the signature of a bimodal distribution.

Analysis

Throughput scales, but the scaling is partially artefactual. RPS grows 0.02 → 0.07 → 0.19, a 9.5× improvement for 4× replicas. Super-linear scaling in a pure compute system would be suspicious; here the explanation is visible directly in the latency distribution.

The bimodal latency is the finding. At r=2 and r=4 the distribution has two distinct peaks: a fast mode around 4–9 s and a slow mode at 100 s or more. The mechanism is architectural. ALB target-group round-robin distributes incoming `/query` requests approximately evenly across tasks. Only the original r=1 task has the 1 k chunks in its per-task `qdrant`; tasks created by `desired_count=2/4` start with empty qdrant. A request routed to an empty replica retrieves nothing, the LLM is prompted with no context, and it returns "I do not know" after generating only a few tokens — roughly 4–9 s of work. A request routed to the seeded replica follows the full RAG path with real context, and it waits in that single replica's LLM queue, completing in 100+ s.

So the P50 collapse (170 → 4 s) is the empty-index fast path dominating the median as the ratio of empty replicas grows. The stubborn P95 (260 → 172 s) is the seeded replica's queue, which got only mildly relief because the slow path's effective concurrency per replica barely changed.

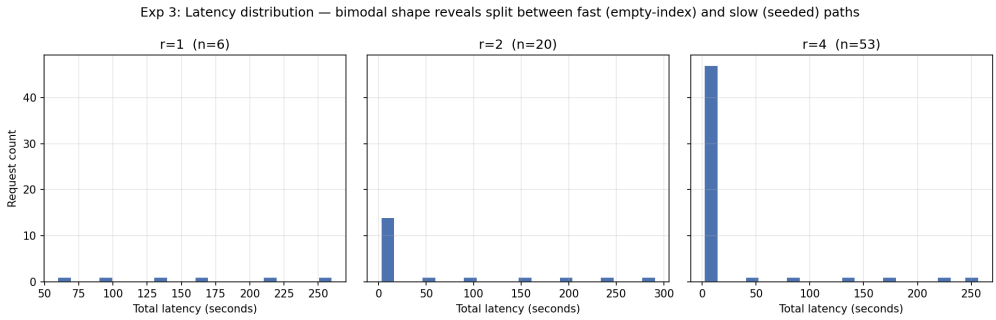

Figure 6: Per-request latency histograms. r=2 and r=4 show two peaks: a fast ~4–9 s mode and a slow mode at 100 s+.

The correct conclusion is not "scaling works." It is "stateless-compute horizontal scaling works for throughput in this design, but it does not deliver the latency improvement real users would experience, because the vector store is coupled to the ECS task's ephemeral storage." A production version would require decoupling the index: external managed Qdrant, an EFS-mounted shared volume, or a dedicated `qdrant` service with its own Fargate task that every api/llm replica reads from. Once state is shared, the LLM tier can scale horizontally without asymmetry, and P50/P95 should converge downward together.

Limitations. (1) The 1 vCPU per-task downsize means absolute latencies here are slower than Exp 1/2 — the comparisons across r are still valid but the numbers should not be compared across experiments without this caveat. (2) Round-robin is an assumption about ALB behaviour under the default target-group attributes; with sticky sessions the asymmetry would manifest differently. (3) 5 min at c=10 with mean latencies 20 s yields small sample sizes (n = 6, 20, 53), so percentiles at the r=1 end are estimated from few data points.

Cross-experiment findings

Three experiments, one coherent system story:

1. **LLM inference is the bottleneck.** Exp 1 showed it at 99.8 % of total latency at every concurrency. Any scaling work that does not target LLM is second-order.
2. **Retrieval is effectively free.** Exp 2 showed retrieve adding only 5 ms going from 1 k to 100 k chunks. HNSW delivers O(log N) in practice. Memory footprint is negligible.
3. **Compute-only horizontal scaling is a partial solution.** Exp 3 showed that replicating the tier does raise throughput, but with per-task state the latency distribution fractures into fast (empty-state) and slow (seeded-state) paths. True horizontal scaling requires decoupling the vector store from the compute task.

Recommended next evolution: (a) externalize `qdrant` to its own service, (b) keep `api/embed/llm` as stateless replicable tasks behind the existing ALB target group, (c) scale LLM replicas aggressively (Exp 1), (d) leave retrieval headroom alone until >1 M chunks (Exp 2). Concrete architectural guidance grounded in measurement, not speculation — which is what these experiments were designed to produce.